
UNIVERSIDAD SIMÓN BOLÍVAR
FACULTAD DE INGENIERÍA DE SISTEMAS

MANUAL DE CÓDIGO Y ARQUITECTURA

Matemáticas Discretas para Ingeniería de Sistemas

Objeto Virtual de Aprendizaje (OVA) Interactivo

DOCUMENTO TÉCNICO OFICIAL

Cúcuta, Colombia • 2026

Objeto Virtual de Aprendizaje (OVA): Matemáticas Discretas

Manual De Código Y Arquitectura

Este manual explica la organización interna del código para que el proyecto pueda mantenerse, ampliarse y revisarse con facilidad.

1. Arquitectura General

El OVA es una Single Page Application estática. Sus piezas principales son:

- `index.html`: estructura base, contenedores principales, barra lateral, carga de MathJax y referencias a CSS/JS.
- `styles.css`: sistema visual, diseño responsivo, modo oscuro, tarjetas, pestañas, modales, certificado y estilos de laboratorios.
- `app.js`: datos académicos, renderizado de vistas, navegación, estado, laboratorios, juegos, audio, evaluación final y certificado.
- `generate_pdfs.js`: conversor de manuales Markdown a PDF.
- `generate_h5p_package.js`: generador del paquete `.h5p` importable en Lumi.

No hay backend. Todo el comportamiento ocurre en el navegador.

2. Estado Y Persistencia

El estado del estudiante vive en `localStorage` con la clave:

```
const STORAGE_KEY = "ova-discretas-progress-v1";
```

La función `defaultState()` define la estructura base:

```
{
  completed: {},
  quizAnswers: {},
  gameAnswers: {},
  extraActivities: {},
  finalAnswers: {},
  soundOn: true,
  finalScore: null
}
```

`loadState()` mezcla el estado guardado con el estado por defecto para soportar futuras versiones sin romper datos anteriores. `saveState()` serializa el progreso después de interacciones importantes.

3. Fuente De Contenido

El contenido principal está en ``app.js``:

- ``modules``: datos de los seis módulos, objetivos, ejemplos, PDFs, outcomes y preguntas.
- ``moduleDeepDives``: contenido ampliado por tema.
- ``moduleSystemsFocus``: relación con Ingeniería de Sistemas.
- ``moduleLabChallenges``: retos guiados de laboratorio.
- ``moduleGames``: minijuegos de clasificación.
- ``interactiveActivities``: actividades extra por módulo.
- ``moduleStudyGuides``: definiciones, procedimiento, ejemplo y práctica.
- ``finalQuestions``: banco de evaluación final.

Para editar teoría, preguntas o ejemplos, normalmente basta con modificar estos objetos sin tocar el motor de renderizado.

4. Renderizado De Vistas

Las vistas principales son:

- ``renderOverview()``: portada, tarjetas de módulos, progreso y acceso a evaluación final.
- ``openModule(moduleId)``: renderiza un módulo completo con pestañas.
- ``renderFinalEvaluation()``: evaluación integradora, calificación y certificado.
- ``renderPlayground()``: playground global de pruebas libres.

El diseño evita duplicar HTML manualmente: las vistas se reconstruyen desde los datos del módulo y el estado guardado.

5. Sistema De Pestañas

``initTabs(root, currentModuleId)`` conecta las pestañas internas de un módulo:

- Teoría y guía.
- Laboratorio.
- Evaluación y juego.
- Pruebas libres.

Al cambiar de pestaña se detiene la narración activa, se actualiza el panel visible, se refresca el simulador correspondiente y se vuelve a ejecutar el renderizado matemático.

6. Renderizado Matemático

``renderMath(root)`` intenta usar MathJax v3 cuando está disponible. Si el CDN no responde, ejecuta ``renderLocalLatex(root)``, un fallback local para que símbolos frecuentes como fracciones, binomiales, conjuntos y expresiones lógicas sigan legibles.

Funciones auxiliares:

- ``inlineMath(tex)``: devuelve notación LaTeX en línea.
- ``displayMath(tex)``: devuelve bloque matemático.
- ``latexToHtml(tex)``: transforma notación común a HTML local.

7. Laboratorios

``renderLab(moduleId)`` funciona como despachador. Cada tema tiene su función:

- ``renderLogicLab()``
- ``renderSetsLab()``
- ``renderStringsLab()``
- ``renderProbabilityLab()``
- ``renderBinomialLab()``
- ``renderGraphLab()``

Las funciones ``update...Lab()`` calculan resultados y refrescan la interfaz. El patrón es siempre el mismo: leer inputs, validar, calcular, pintar resultados y llamar a ``renderMath()`` si aparecen fórmulas.

8. Juegos, Actividades Y Evaluaciones

``renderGame(moduleId)`` monta minijuegos de clasificación. ``checkGame(moduleId)`` valida respuestas y guarda progreso.

``renderExtraActivity(moduleId)`` despacha actividades por tipo:

- clasificación,
- selector de evento,
- validador de cadenas,
- juez euleriano.

``createQuestionCard(question, key, onAnswer)`` es el componente reutilizable para autoevaluaciones y evaluación final. Maneja selección, comprobación, reintento, solución y persistencia.

9. Audio Y Voz

El OVA usa dos sistemas:

- Web Audio API: genera efectos sonoros pequeños en memoria, sin depender de archivos externos.
- SpeechSynthesis API: lee textos teóricos y retroalimentaciones en voz alta.

``stopSpeaking()`` cancela narraciones activas al cambiar de vista o pestaña.

10. Certificado

Cuando el estudiante obtiene 80% o más en la evaluación final:

1. ``gradeFinal()`` calcula el puntaje.
2. ``showCertificateForm()`` solicita nombre y apellido.
3. ``showCertificateModal(name)`` muestra el certificado imprimible.
4. ``window.print()`` permite guardar o imprimir como PDF.

El nombre se guarda localmente para reutilizarlo si el estudiante vuelve a abrir el certificado.

11. Generador H5P

`generate\_h5p\_package.js` crea un `.h5p` con estructura ZIP válida:

```
h5p.json
content/content.json
H5P.OvaMatematicasDiscretas-1.0/library.json
H5P.OvaMatematicasDiscretas-1.0/semantics.json
H5P.OvaMatematicasDiscretas-1.0/scripts/ova.js
H5P.OvaMatematicasDiscretas-1.0/styles/ova.css
```

El contenido se construye desde datos equivalentes a los seis módulos del OVA. La versión H5P es editable/importable en Lumi y complementa a la app web completa.

12. Cómo Agregar Un Nuevo Módulo

Para agregar un módulo nuevo:

1. Crear un objeto en ``modules`` con ``id``, ``number``, ``title``, ``summary``, ``outcomes``, ``example`` y ``quiz``.
2. Agregar contenido en ``moduleDeepDives``.
3. Agregar aplicación profesional en ``moduleSystemsFocus``.
4. Crear reto en ``moduleLabChallenges``.
5. Agregar minijuego en ``moduleGames``.
6. Agregar guía en ``moduleStudyGuides``.
7. Implementar un laboratorio nuevo y enlazarlo en ``renderLab(moduleId)``.
8. Si requiere pruebas libres, agregar HTML y eventos en el sistema de playgrounds.

El patrón recomendado es mantener el contenido en objetos de datos y dejar la lógica de renderizado como motor reutilizable.

